WEEK 11

Feynman's Vision and Deutsch's Algorithm

27, 29 October · Objectives 6, 8

Feynman's 1981 observation was that simulating quantum systems on classical hardware costs exponentially, so the machine should be quantum. Deutsch turned that into the first algorithm with a provable quantum advantage — small, artificial, and containing the complete pattern every later algorithm follows.

Feynman's argument

Representing n qubits classically requires 2^n complex amplitudes. Fifty spins need $2^{50} \approx 10^{15}$ numbers; a hundred exceeds the atom count of any storage medium you could build. Chemistry and materials science therefore hit an exponential wall on classical hardware.

Feynman's proposal, at MIT's 1981 *Physics of Computation* conference: use a controllable quantum system as the simulator. "Nature isn't classical, dammit, and if you want to make a simulation of nature, you'd better make it quantum mechanical."

The framing that holds up: this is a **native-execution** argument, not an acceleration argument. You are not speeding up a classical algorithm; you are running the problem on hardware whose state space has the same structure as the problem's. Quantum chemistry remains the most credible near-term application for exactly this reason — the mapping is natural rather than contrived.

The oracle model

Deutsch's problem: given a black-box function f: {0,1} → {0,1}, decide whether it is **constant** (f(0) = f(1)) or **balanced** (f(0) ≠ f(1)).

Four possible functions; two constant, two balanced. Classically you must evaluate f twice — one evaluation leaves both cases possible. **One query is provably insufficient.**

The quantum oracle must be unitary, hence reversible. The standard construction:

```
U_f |x⟩|y⟩ = |x⟩|y ⊕ f(x)⟩
```

XOR onto an ancilla, preserving x. This is reversible (apply twice to get the identity) and is the generic recipe for embedding any classical function in a quantum circuit.

> **Analogy boundary — the oracle as a function call.** U_f is a callable with a fixed signature, and query counting is a clean complexity measure. Two boundaries. First, U_f must be **reversible**, so it cannot discard inputs — every classical function needs the ancilla embedding, and that is a real cost. Second, and more important: you may call it on a **superposition of arguments**, which has no software analogue. That is not "calling it many times fast." It is one invocation whose input is a single state that happens not to be a basis state, and the distinction matters for reasoning about cost.

Phase kickback

The mechanism that makes the algorithm work. Set the ancilla to $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$:

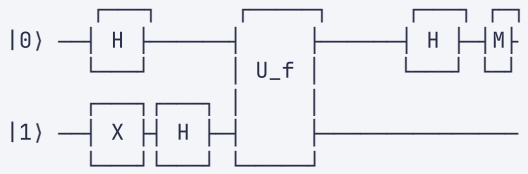
```
U_f |x⟩|-⟩ = |x⟩ · (|0 ⊕ f(x)⟩ - |1 ⊕ f(x)⟩)/√2
```

If f(x) = 0 the ancilla is unchanged. If f(x) = 1 the two terms swap, which is an overall sign flip. Combining:

```
U_f |x⟩|-⟩ = (-1)^{f(x)} |x⟩|-⟩
```

The ancilla is untouched. The function's **output has become a phase on the input register**. This is the same effect seen with CNOT in Week 8, and it is the workhorse of quantum algorithms: it converts a value you cannot read into a phase you can interfere.

Deutsch's algorithm



Step by step. After the Hadamards, the input register is $|+\rangle$ and the ancilla is $|-\rangle$:

```
(|0⟩ + |1⟩)/√2 ⊗ |-⟩
```

Apply U_f, using phase kickback:

```
→ ((-1)^{f(0)}|0⟩ + (-1)^{f(1)}|1⟩)/√2 ⊗ |-⟩
```

Factor out the global phase (−1)^{f(0)}, which is unobservable:

```
→ (|0⟩ + (-1)^{f(0)⊕f(1)}|1⟩)/√2
```

The relative phase now encodes **f(0) ⊕ f(1)** — precisely the quantity asked for. Apply H:

```
f(0) ⊕ f(1) = 0  (constant)  →  |+⟩ → |0⟩
f(0) ⊕ f(1) = 1  (balanced)  →  |-⟩ → |1⟩
```

Measure. **One query, deterministic answer.**

Note what happened: at no point did you learn f(0) or f(1) individually. The algorithm extracts a *global property* of the function while leaving its values inaccessible. That trade is the essence of quantum algorithms — you get one bit about the whole function, not the function.

> **Analogy boundary — "evaluating both inputs at once."** The standard gloss is that the superposition evaluates f on 0 and 1 simultaneously. This is misleading in a way that will cost you later. Both amplitudes evolve, yes — but you can never read both results. The algorithm works because the **final Hadamard interferes them**, and the interference is constructive exactly when the answer is one bit. Drop the H and you get a random bit and learn nothing. Parallelism is not the resource; **engineered interference** is.

Deutsch–Jozsa

The n-bit generalization. Given $f: \{0,1\}^n \to \{0,1\}$, promised to be either constant or balanced (exactly half the inputs map to 1), decide which.

Classically, worst case requires $2^{n-1} + 1$ queries for certainty. Quantum: **one query**, by the same H → U_f → H structure on n qubits. An exponential separation in exact query complexity.

The caveat that matters for Week 16's honesty: with randomization, a classical algorithm decides this with high probability in $O(1)$ queries — sample a few inputs and see if they differ. The exponential gap holds only for *deterministic exact* classical computation. Deutsch– Jozsa proves quantum computers are

provably different, not that they are practically useful for this task. It is a proof of concept, and it should be described as one.

Key takeaways

- Feynman's argument is about native execution: quantum systems cost exponential classical memory, so simulate them on quantum hardware.
- Reversible oracles use the XOR embedding $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$, requiring an ancilla.
- Phase kickback with the ancilla in $|-\rangle$ converts function output into an input-register phase: $U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$.
- Deutsch decides constant vs. balanced in one query; the final Hadamard converts phase into a measurable bit.
- Deutsch–Jozsa gives exponential separation against *deterministic* classical algorithms only. Randomized classical algorithms close the gap.

Self-check

Q1 *(Objective 6)* — Remove the final Hadamard from Deutsch's algorithm and measure immediately after U_f. What do you observe, and what does this establish about the source of quantum advantage?

Q2 *(Objectives 6, 8)* — Deutsch–Jozsa needs one query where deterministic classical needs $2^{n-1}+1$. Explain why this does not demonstrate practical advantage, and identify the property a problem needs for the separation to survive randomization.

Check your answers

A1 — Without the final H you measure the state $(|0\rangle \pm |1\rangle)/\sqrt{2}$ in the computational basis and get **0 or 1 with equal probability, regardless of f.** Both the constant and balanced cases yield a uniform random bit, and the outcome carries no information about the function.

The reason: the answer is encoded in the **relative phase** between $|0\rangle$ and $|1\rangle$, and computational-basis measurement is blind to relative phase — $|\langle 0|\psi\rangle|^2 = |\langle 1|\psi\rangle|^2 = 1/2$ for both $|+\rangle$ and $|-\rangle$. The information is present in the state and inaccessible to that measurement.

This establishes that quantum advantage comes from **interference, not superposition.** The superposition existed and the oracle was queried on it, so if parallel evaluation were the resource, the answer would be available. It is not. The Hadamard rotates the {|+⟩, |−⟩} basis — in which the two cases are orthogonal and perfectly distinguishable — onto the computational basis the hardware can read. Every algorithm in this course ends with such a rotation: Grover's diffusion operator and Shor's inverse QFT are both this step generalized.

A2 — Two independent reasons.

The problem is artificial. The promise that f is *either* constant *or* exactly balanced — never anything else — is not a property real functions have. Deutsch–Jozsa solves a problem constructed to be solvable, which makes it a proof of separation rather than an application.

Randomization closes the gap. Sample two random inputs. If they differ, f is balanced — certain. If they agree, guess constant. For a balanced f, each pair agrees with probability $\approx 1/2$, so k samples give error $\leq 2^{-k}$. Ten queries yield error under 0.1%, independent of n. The exponential separation exists only against algorithms required to be *exactly* correct with certainty, which is an unusual and rarely-needed requirement.

For a separation to survive randomization, the problem needs structure that random sampling cannot exploit — typically a **global, periodic, or algebraic property** invisible in any small set of evaluations. Simon's problem is the clean example: it has a hidden XOR-mask period, requires exponentially many queries even for randomized classical algorithms, and directly inspired Shor's algorithm. Shor's period-finding is the same idea applied to modular exponentiation, and it is why factoring — not Deutsch–Jozsa — is the result that changed cryptography.